Adaptive Communication Network with Failure Recovery

SDN + UDP Socket Programming (Jackfruit Mini Project, Problem 30)

Deliverable 1 Report | Computer Networks (UE25CS243A)

Field	Details
Student name(s)	MUNNANGI VIJAYA ADITHYA NIRANJAN BASAPPA BENAKANAVAR PRAJWAL MANJUNATH MALLAPPANAVAR
SRN (s)	PES1UG25CS317 PES1UG25CS343 PES1UG25CS370
Section / Faculty	F section / **RADHIKA M HIRANNAIAH**
GitHub repository	https://github.com/Niranjan728/adaptive-udp-sdn
Controller / Tools	Ryu (OpenFlow 1.3), Mininet 2.3.0, Open vSwitch 3.7.1, Python 3

1. Problem statement and objectives

Networks fail: links go down, and a plain UDP application gives no delivery guarantee when they do. This project builds a resilient UDP communication framework in which an SDN controller detects network failures and dynamically reroutes the application's traffic through an alternate path, while the application itself detects the outage and recovers lost data.

Objectives (from the problem statement):

- Implement UDP communication using low-level sockets.
- Create multiple network paths between the communicating hosts.
- Monitor topology and link status.
- Simulate link/path failures.
- Detect failures dynamically and reroute application traffic.
- Implement application-level recovery.
- Measure recovery time, packet loss and latency (the measurement part is the focus of Deliverable 2).

2. System architecture

Two UDP clients (h1, h3) send messages to a UDP server (h2) across five OpenFlow switches. Two disjoint paths exist: a short primary path through s2 and a longer, slower backup path through s3 and s5. A Ryu controller is connected to every switch.

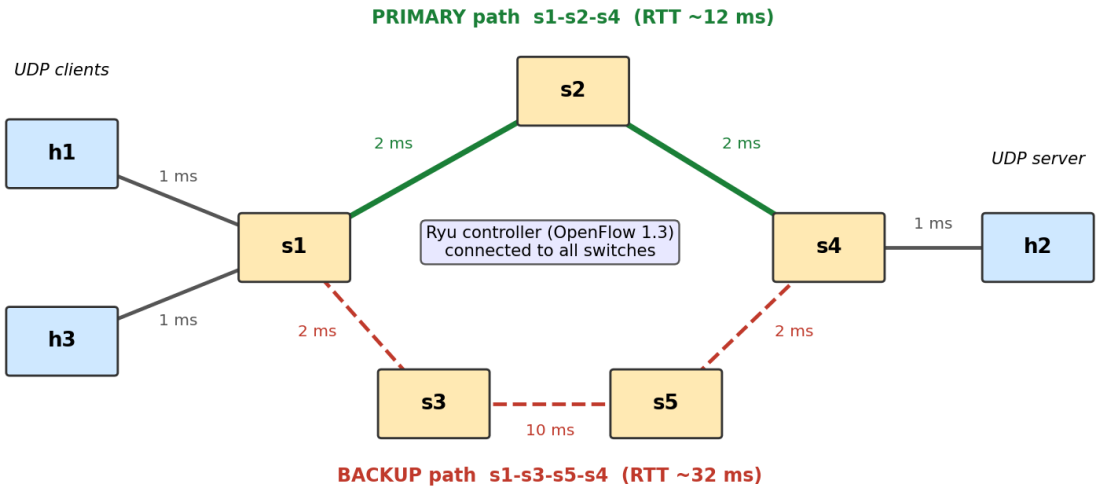


Figure 1. Mininet topology with primary (green) and backup (red) paths. Link delays are shown.

2.1 Components

Component	Role	File
UDP server (h2)	Receives DATA/PROBE datagrams, replies with ACK/PROBE_ACK, detects duplicates, logs arrivals. Receiver thread plus worker threads.	src/server.py
UDP client (h1, h3)	Sends numbered messages at a fixed rate, tracks ACKs, retransmits, detects outages, probes and resends after recovery.	src/client.py
Protocol	16-byte header: type, client id, attempt, sequence number, send timestamp.	src/protocol.py
Ryu controller	Topology discovery, failure detection, shortest-path routing, ARP proxy, port-statistics monitoring, event log.	controller/adaptive_router.py
Mininet topology	Five OVS switches, three hosts, links with bandwidth and delay (TCLink).	topology/topo.py

2.2 Communication flow

1. A client sends DATA(seq) to the server's IP (10.0.0.2, UDP port 5000); the server answers with ACK(seq).
2. Packets follow flow rules installed by the controller: match on destination IP, output to the next hop of the shortest path.
3. The controller keeps a graph of the network built from LLDP link discovery and switch port-status messages.
4. When a link fails, the controller removes it from the graph, recomputes the paths and overwrites the flow rules, so traffic moves to the backup path.
5. If packets are lost during the switch-over, the client's recovery logic retransmits them.

2.3 Expected network behaviour

Normally traffic from h1 and h3 to h2 uses the primary path s1→s2→s4 (round-trip time about 12 ms). If the link s2–s4 fails, the controller should reroute over s1→s3→s5→s4 (about 32 ms round trip) within milliseconds, and the application should lose few or no messages. When the link returns, the controller moves traffic back to the primary path.

3. UDP socket implementation

The application uses plain UDP sockets (socket.SOCK_DGRAM) with no reliability library. Reliability is built at the application layer on purpose.

3.1 Message format

Field	Size	Meaning
type	1 byte	DATA, ACK, PROBE or PROBE_ACK
client id	1 byte	Identifies the client so one server can serve many
attempt	2 bytes	Transmission attempt number (1 = first try)
seq	4 bytes	Per-client sequence number (0 for probes)
send timestamp	8 bytes	Sender time; echoed unchanged in the ACK so the client measures the round-trip time of the exact attempt that was acknowledged

3.2 Server

One receiver thread reads datagrams and puts them on a queue; a pool of worker threads parses them, detects duplicate retransmissions per (client id, sequence number), sends the ACK and logs the arrival time. Malformed datagrams are ignored. The log is written when the server stops (SIGINT/SIGTERM).

3.3 Client and application-level recovery

The client generates one message every 20 ms (50 messages per second by default). It works as a small state machine:

- **NORMAL:** every message is tracked until acknowledged; unacknowledged messages are retransmitted after a 0.15 s timeout, at most 5 times.
- **Outage detection:** if messages are pending and no ACK has arrived for 0.4 s, the client switches to RECOVERING.
- **RECOVERING:** new messages are buffered (no retransmission storm) and a small PROBE datagram is sent every 0.1 s.
- **Back to NORMAL:** the first PROBE_ACK shows the path works again; all unacknowledged and buffered messages are re-sent in order.

A switch (--recovery off) turns the client into a plain fire-and-forget UDP sender, which is used as the baseline in Deliverable 2.

3.4 Socket-layer test (without the network)

A test script (tests/test_local_recovery.py) places a UDP proxy between client and server that drops all packets for 2 seconds. With recovery enabled no message was lost; with recovery disabled all 100 messages sent during the blackout were lost.

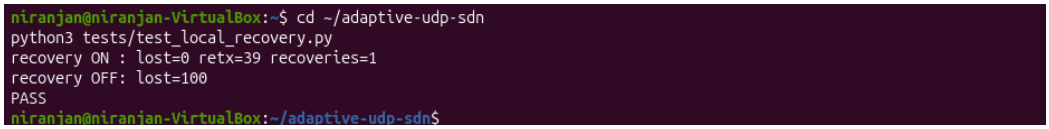


Figure 2. Local socket-layer test: recovery ON loses 0 messages, recovery OFF loses 100.

4. Mininet and SDN implementation

4.1 Topology

The topology (topology/topo.py) has hosts h1 (10.0.0.1), h3 (10.0.0.3) and h2 (10.0.0.2) with fixed MAC addresses, five Open vSwitch switches using OpenFlow 1.3, and TCLink links with bandwidth 10 Mbit/s. The backup path is deliberately slower (the s3–s5 link has 10 ms delay), so the latency change after a failover is visible.

4.2 Controller (Ryu, OpenFlow 1.3)

- **Topology discovery:** LLDP link events (ryu.topology, started with --observe-links) build the link graph.
- **Fast failure detection:** OFPPortStatus (LINK_DOWN) messages from the switches; LLDP timeout is a fallback.
- **Routing:** breadth-first shortest path (hop count) on the current graph; one destination-IP flow per switch and host, at priority 10.
- **ARP proxy:** the topology contains loops, so ARP is answered by the controller and unknown requests are flooded only on host-facing ports, which avoids broadcast storms.
- **Monitoring and logging:** port counters are polled every second (port_stats.csv) and every event is time-stamped (controller_events.csv).

4.3 Evidence: network up and flow rules installed

After starting the controller and the network, pingall shows full connectivity, and the controller has installed one rule per host on every switch.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
```

Figure 3. pingall: all three hosts reach each other, 0% dropped.

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=493.395s, table=0, n_packets=1076, n_bytes=64560, priority=65535,dl_dst=01:80:c2:00:00:0e,dl_type=0x88cc actions=CONTROLLER:65535
 cookie=0x0, duration=467.381s, table=0, n_packets=12048, n_bytes=698944, priority=10,ip,nw_dst=10.0.0.1 actions=output:"s1-eth1"
 cookie=0x0, duration=467.381s, table=0, n_packets=12048, n_bytes=1469632, priority=10,ip,nw_dst=10.0.0.2 actions=output:"s1-eth3"
 cookie=0x0, duration=467.381s, table=0, n_packets=4, n_bytes=392, priority=10,ip,nw_dst=10.0.0.3 actions=output:"s1-eth2"
 cookie=0x0, duration=493.943s, table=0, n_packets=75, n_bytes=7256, priority=0 actions=CONTROLLER:65535
```

Figure 4. Flow rules on s1 before the failure: traffic for 10.0.0.2 (the server) leaves via s1-eth3, the port towards s2 (primary path). (Captured in an earlier run of the same topology.)

5. Socket–SDN integration

The UDP application and the SDN network were tested together: the client was running and sending to the server while the link s2–s4 was taken down with the Mininet command “link s2 s4 down”.

```
routes_updated reason=host_learned flows=10 took_ms=1.21 paths=[10.0.0.1->10.0.0.2:s1>s2>s4]
host_learned 10.0.0.3 00:00:00:00:00:03 at s1:2
routes_updated reason=host_learned flows=15 took_ms=4.37 paths=[10.0.0.1->10.0.0.2:s1>s2>s4 10.0.0.1->10.0.0.3:s1 10.0.0.2->10.0.0.3:s4>s2>s1]
port_status s4 port=2 down=False
port_status s2 port=2 down=False
port_status s2 port=2 down=True
link_down_detected s2:2 via port_status
routes_updated reason=link_down flows=15 took_ms=5.08 paths=[10.0.0.1->10.0.0.2:s1>s3>s5>s4 10.0.0.1->10.0.0.3:s1 10.0.0.2->10.0.0.3:s4>s5>s3>s1]
port_status s4 port=2 down=True
port_status s4 port=2 down=True
```

Figure 5. Controller log: port_status reports the port down, link_down_detected follows, and routes_updated changes the h1→h2 path from s1>s2>s4 to s1>s3>s5>s4 (5.08 ms to recompute and install all 15 flows).

```
mininet> link s2 s4 down
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=358.944s, table=0, n_packets=758, n_bytes=45480, priority=65535,dl_dst=01:80:c2:00:00:0e,dl_type=0x88cc actions=CONTROLLER:65535
 cookie=0x0, duration=126.287s, table=0, n_packets=12497, n_bytes=725846, priority=10,ip,nw_dst=10.0.0.1 actions=output:"s1-eth1"
 cookie=0x0, duration=126.286s, table=0, n_packets=12629, n_bytes=1533730, priority=10,ip,nw_dst=10.0.0.2 actions=output:"s1-eth4"
 cookie=0x0, duration=126.286s, table=0, n_packets=4, n_bytes=392, priority=10,ip,nw_dst=10.0.0.3 actions=output:"s1-eth2"
 cookie=0x0, duration=358.968s, table=0, n_packets=103, n_bytes=27024, priority=0 actions=CONTROLLER:65535
mininet>
```

Figure 6. Flow rules on s1 after the failure: traffic for 10.0.0.2 now leaves via s1-eth4, the port towards s3 (backup path).

	Before failure	After failure
Path h1 → h2	s1 > s2 > s4	s1 > s3 > s5 > s4
s1 output port for 10.0.0.2	s1-eth3 (towards s2)	s1-eth4 (towards s3)
Controller event	-	port_status, link_down_detected, routes_updated

When the link is brought back up (link s2 s4 up) the controller logs link_up and switches the path back to s1>s2>s4, so both failure and recovery of the network are handled.

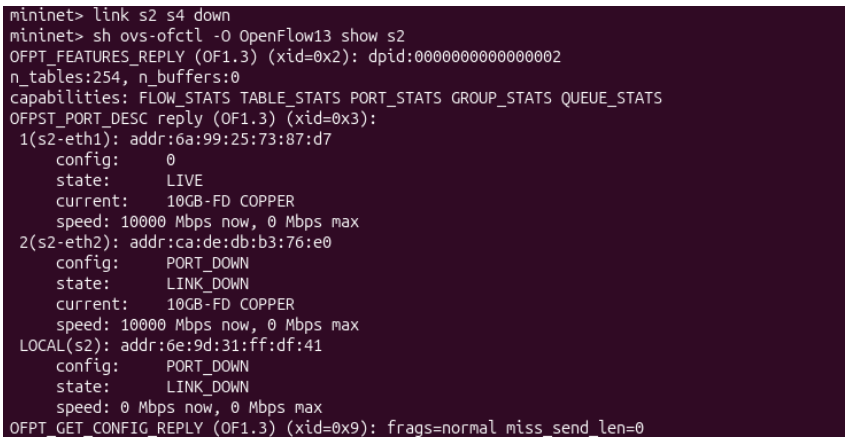


Figure 7. OpenFlow port description of s2 after the failure: port s2-eth2 reports PORT_DOWN / LINK_DOWN (the failure is real, not only logged).

6. Initial testing and results

ID	Test	Observed result	Evidence
T1	UDP socket layer without network	Recovery ON: 0 lost; OFF: 100 lost; PASS	Figure 2
T2	Connectivity (pingall)	0% dropped (6/6 received)	Figure 3
T3	Controller installs flows	Rule per host on s1; 10.0.0.2 via s1-eth3	Figure 4
T4	Failure detection	link_down_detected via port_status; routes recomputed in 5.08 ms	Figure 5
T5	Reroute to backup path	10.0.0.2 via s1-eth4; path s1>s3>s5>s4	Figures 5, 6
T6	UDP traffic during the failure	4500 sent, 4500 acknowledged, 0 lost, 36 retransmissions	Figure 8
T7	Link restored	Controller returns the path to s1>s2>s4	Controller log

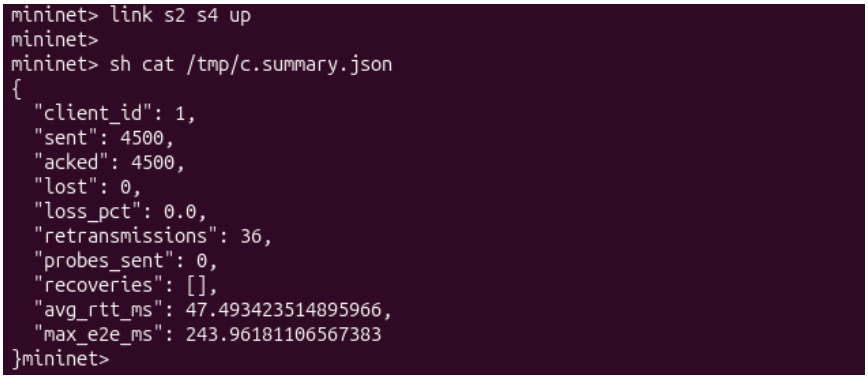


Figure 8. Client summary after a 90-second run (4500 messages) during which the primary link failed and was restored: no message was lost.

Interpretation. All 4500 messages were acknowledged although the primary link failed during the run. The 36 retransmissions include messages sent in the short interval between the failure and the controller's route update, which the client resent over the backup path (a few retransmissions also occur without any failure, for example 17 in an earlier failure-free run, because of queueing delay on the 10 Mbit/s links). The average round-trip time of 47 ms and the maximum message latency of 244 ms include retransmitted messages and time spent on the longer backup path. The client's outage detector (0.4 s) did not trigger in this run, which suggests the interruption was shorter than 0.4 s, so the SDN reroute alone restored the service quickly. Precise recovery times over repeated runs will be measured in Deliverable 2.

7. How to reproduce

Environment: Ubuntu VM (VirtualBox), Mininet 2.3.0, Open vSwitch 3.7.1. Ryu is installed in a separate Python 3.9 environment because the system Python is too new for Ryu.

7.1 Installation

```
sudo apt install mininet openvswitch-switch curl -y
curl -LsSf https://astral.sh/uv/install.sh | sh
source $HOME/.local/bin/env
uv venv --python 3.9 ~/ryu-env
uv pip install --python ~/ryu-env/bin/python "setuptools<68" wheel pbr
uv pip install --python ~/ryu-env/bin/python --no-build-isolation \
    ryu "eventlet==0.30.2" "dnspython<2"
```

7.2 Running the demo (two terminals)

Terminal 1: controller

```
cd ~/adaptive-udp-sdn
ROUTING_MODE=dynamic RESULTS_DIR=results/demo \
  ~/ryu-env/bin/ryu-manager --observe-links controller/adaptive_router.py
```

Terminal 2: network and application

```
cd ~/adaptive-udp-sdn
sudo python3 topology/topo.py --cli
mininet> pingall
mininet> h2 python3 src/server.py --log /tmp/s.csv &
mininet> h1 python3 src/client.py --server 10.0.0.2 --duration 90 --log /tmp/c.csv &
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
mininet> link s2 s4 down
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
mininet> link s2 s4 up
mininet> sh cat /tmp/c.summary.json
```

Cleanup after a run: exit the Mininet prompt, run "sudo mn -c", and stop the controller with Ctrl+C. The socket-layer test runs without Mininet: python3 tests/test_local_recovery.py.

8. Plan for Deliverable 2

The repository already contains the scripts for the full evaluation; they will be run and their results documented in Deliverable 2:

- **Baseline comparison:** a static-routing controller (ignores link failures) against the dynamic controller, each with and without application-level recovery (four configurations).
- **Challenging scenarios:** double failure (no path available for a while) and link flapping, with automated runs and repetitions.

- **Metrics:** detection time, reroute time, recovery time, packet loss, round-trip time before and after failover, retransmissions; results shown as tables and plots (analysis/analyze.py).
- **Documentation and viva:** final README with results, architecture and design decisions.

9. Repository structure

Folder	Files	Purpose
src/	protocol.py, server.py, client.py	UDP application
controller/	adaptive_router.py	Ryu controller (dynamic and static modes)
topology/	topo.py	Mininet topology
experiments/	run_experiment.py, run_all.sh	Automation (Deliverable 2)
analysis/	analyze.py	Tables and plots (Deliverable 2)
tests/	test_local_recovery.py	Socket-layer test
README.md	-	Setup and run instructions